The Levi-Civita tensor ϵ

three aspects of the totally antisymmetric tensor ϵ:

1. its relation to the determinant of a matrix
2. its role as invariant tensor of so(n), in the example of so(3)
3. using it as example of how symmetries can provide efficient computer storage

In[2400]:=
```
Remove["Global`*"]
```

In[2401]:=
```
ϵ = LeviCivitaTensor[3, List]
```

Out[2401]=
$$\begin{pmatrix} \{0,0,0\} & \{0,0,1\} & \{0,-1,0\} \\ \{0,0,-1\} & \{0,0,0\} & \{1,0,0\} \\ \{0,1,0\} & \{-1,0,0\} & \{0,0,0\} \end{pmatrix}$$

In[2402]:=
```
ϵ[[1, 2, 3]]
```

Out[2402]=
1

(see below what the option "`List`" means here).
Another way to represent these numbers, as "3 x 3 x 3 matrix" (think depth):

In[2403]:=
```
Table[MatrixForm[ϵ[[i]]], {i, 1, 3}]
```

Out[2403]=
$$\left\{\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 0 & -1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}\right\}$$

1. the relationship to the determinant of a matrix.
take e.g. the 3 rows M_i (i = 1,2,3) of a 3x3 matrix M. Form
$M_1 \otimes M_1 \otimes M_3 \otimes \epsilon$
which can be viewed as a tensor with 6 indices:

In[2404]:=
```
M = Array[m, {3, 3}]
MMMϵ = M[[1]]⊗M[[2]]⊗M[[3]]⊗ϵ;
MMMϵ[[1, 2, 3, 1, 2, 3]]
```

Out[2404]=
$$\begin{pmatrix} m(1,1) & m(1,2) & m(1,3) \\ m(2,1) & m(2,2) & m(2,3) \\ m(3,1) & m(3,2) & m(3,3) \end{pmatrix}$$

Out[2406]=
$m(1,1)\, m(2,2)\, m(3,3)$

You can inspect the $3^6 = 729$ elements of `MMMϵ` by modifying the [[`1,2,3,1,2,3`]] above.
(More below about using symmetries to restrict computer storage. Here that aspect is not important: 729 element is no problem to *handle*, just clumsy to *display*.)
Now **contract** indices 1 ↔ 4, 2 ↔ 5, 3 ↔ 6. This creates det(M):

In[2407]:=
```
Det[M] === TensorContract[MMMϵ, {{1, 4}, {2, 5}, {3, 6}}]
```

Out[2407]=
True

With Mathematica, it's easy to check that this is not special to 3 x 3 matrices, but works equally well e.g. for 4 x 4 matrices:

In[2408]:=
```
M = Array[m, {4, 4}];
MMMMϵ = M⟦1⟧⊗M⟦2⟧⊗M⟦3⟧⊗M⟦4⟧⊗LeviCivitaTensor[4, List];
Det[M] === TensorContract[MMMMϵ, {{1, 5}, {2, 6}, {3, 7}, {4, 8}}]
```

Out[2410]=
True

For more on contractions, see the Mathematica help page on **TensorContract.**
(The above looks slightly more elegant with the **Notation** package to write m_{ij} for $m(i,j)$, as I do in other files. But by the notation convention "*keep important data big*", I chose not to here.)

2. ϵ as invariant tensor of so(3)

An **invariant tensor** gives zero on the Lie algebra. The standard basis of so(3) is given by these three real antisymmetric matrices L_i:

In[2411]:=
```
L1 = {{0, 0, 0}, {0, 0, -1}, {0, 1, 0}};
L2 = {{0, 0, 1}, {0, 0, 0}, {-1, 0, 0}};
L3 = {{0, -1, 0}, {1, 0, 0}, {0, 0, 0}};
```

These obey the so(3) algebra. For example:

In[2414]:=
```
L1.L2 - L2.L1
```

Out[2414]=
$$\begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

... which is L3.

Now assemble the three L_i to a list L.

As a side remark, it turns out that this list L is minus ϵ:

In[2415]:=
```
L = {L1, L2, L3};
% == -ϵ
```

Out[2416]=
True

The invariant tensor condition to check is $(x\,\epsilon)_{ijk} = 0$ for all $x \in$ so(3).

It's enough to check for x being the 3 generators $(L_m\,\epsilon)_{ijk} = 0$:

$\sum_a L_{m,i,a}\,\epsilon_{a,j,k} + L_{m,j,a}\,\epsilon_{i,a,k} + L_{m,k,a}\,\epsilon_{i,j,a} = 0$

(the index a is of the same type as ijk, so maybe l is better than a, but I'm trying to emphasize the sum, and below I'll do it without having to make a choice).

Explicitly:

In[2417]:=
```
Table[Sum[L⟦m, i, a⟧ × ϵ⟦a, j, k⟧ + L⟦m, j, a⟧ × ϵ⟦i, a, k⟧ + L⟦m, k, a⟧ × ϵ⟦i, j, a⟧,
  {a, 3}], {m, 3}, {i, 3}, {j, 3}, {k, 3}]
```

Out[2417]=
$$\left(\begin{pmatrix}\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}\\ \begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}\\ \begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}\end{pmatrix}\right)$$

OK.

Now, it's in a sense more efficient never to introduce indices *i,j,k,a* at all:

In[2418]:=
```
id = IdentityMatrix[3];
apply[A_, B_, C_] :=
  TensorContract[TensorProduct[A, B, C, ϵ], {{2, 7}, {4, 8}, {6, 9}}];
action[m_] := apply[L⟦m⟧, id, id] + apply[id, L⟦m⟧, id] + apply[id, id, L⟦m⟧];
Table[action[m], {m, 3}]
```

Out[2421]=
$$\left(\begin{pmatrix}\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}\\ \begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}\\ \begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}&\begin{pmatrix}0&0&0\\0&0&0\\0&0&0\end{pmatrix}\end{pmatrix}\right)$$

With this experience, we can do the same for the Kronecker δ:

In[2422]:=
```
δ = IdentityMatrix[3];
id = IdentityMatrix[3];
apply2[A_, B_] := TensorContract[TensorProduct[A, B, δ], {{2, 5}, {4, 6}}];
action2[m_] := apply2[L⟦m⟧, id] + apply2[id, L⟦m⟧];
Table[action2[m], {m, 3}]
```

Out[2426]=
$$\begin{pmatrix}\{0,0,0\}&\{0,0,0\}&\{0,0,0\}\\ \{0,0,0\}&\{0,0,0\}&\{0,0,0\}\\ \{0,0,0\}&\{0,0,0\}&\{0,0,0\}\end{pmatrix}$$

So δ is an invariant tensor of so(3). This is somewhat trivial, since one definition of the Lie group SO(3) to begin with is that the length $\delta_{ij} v^i v^j$ is invariant, which implies that the generators $(L_m)_{ij}$ are antisymmetric in *i,j*.

Note the two different "identity matrices" δ vs. id:
$\delta \in V^* \otimes V^*$, input: two vectors, output: number, $\delta_{ij} v^i v^j$
id $\in$ End(V) = $V \otimes V^*$, input: vector, output: vector id(v) = v

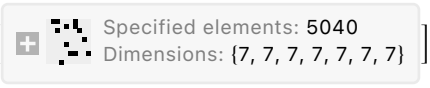
3. ϵ as specified by symmetry properties

In[2427]:=
```
LeviCivitaTensor[7, List];
% // ByteCount
```

Out[2428]=
6 588 536

It's standard procedure in computer science to use **sparse arrays**:

In[2440]:=
```
LeviCivitaTensor[7, SparseArray]
ArrayRules[%][[1 ;; 5]]
% // ByteCount
```

Out[2440]=
SparseArray[]

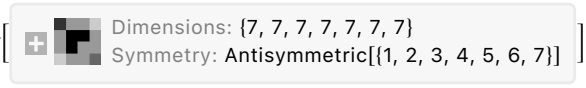
Out[2441]=
$\{\{1, 2, 3, 4, 5, 6, 7\} \to 1, \{1, 2, 3, 4, 5, 7, 6\} \to -1,$
$\{1, 2, 3, 4, 6, 5, 7\} \to -1, \{1, 2, 3, 4, 6, 7, 5\} \to 1, \{1, 2, 3, 4, 7, 5, 6\} \to 1\}$

Out[2442]=
1480

The rules above give the basic idea what a sparse array is: only give nonzero elements. If you are interested in the details, see the help page in Mathematica. But here we will use the next version, not this.

A key advantage of symbolic manipulation is that the essential content of a tensor is stored as **the relations themselves**, not as explicit numbers. Here as `SymmetrizedArray`:

In[2431]:=
```
LeviCivitaTensor[7, SymmetrizedArray]
SymmetrizedArrayRules[%]
% // ByteCount
```

Out[2431]=
SymmetrizedArray[]

Out[2432]=
$\{\{1, 2, 3, 4, 5, 6, 7\} \to 1, \{_, _, _, _, _, _, _\} \to 0\}$

Out[2433]=
736

Note that the rule now only 1. sets the convention of a single element, and 2. implements the vanishing of elements not related to it by the assumed symmetry. These two statements are of course by far not themselves sufficient to determine all 7^7 = 823 543 elements. But if we first impose the complete antisymmetry symmetry, it is.

For more on exactly how this is done, see the Mathematica help pages. But importantly: as a user you don't need to know exactly how it's implemented internally, if you understand the conditions in the first place.
For higher dimension, the comparison goes like this:

In[2434]:=
```
Table[{ByteCount[LeviCivitaTensor[d, SymmetrizedArray]],
  ByteCount[LeviCivitaTensor[d, SparseArray]],
  ByteCount[LeviCivitaTensor[d, List]]}, {d, 2, 9}]
```

Out[2434]=

696	560	144
720	688	336
744	1328	2304
768	5504	25 736
792	36 176	373 432
816	283 528	6 588 536
840	2 581 184	134 217 928
864	26 128 080	3 099 364 120

So if we stopped at 3 dimensions, where the explicit array only uses 336 bytes and the symmetrized one uses 720 bytes, we might conclude that the explicit array is the most efficient! Instead the `SymmetrizedArray` is *by far* the most efficient.

You might think, well 3 billion bytes for 9 dimensions is not so bad? But as you see, it is exponential: on my laptop when I ran this file the first time, I tried 10 dimensions, and it gave an "*out of memory error*" for the explicit list storage. So if you ever needed to do anything that involved 10 variables (dimensions) or more, using an efficient form of storage can be the difference between being feasible and not being feasible.

Note: physicists and engineers can get confused about the **meaning of "dimension"** here. If we never have to go above 3 or at most 3+1 dimensions, who cares? Just think of "dimension" as the number of important variables in your practical problem. For example, for LLMs there can easily be millions of important parameters (or many more)!